

Bridge-RAG: An Abstract Bridge Tree Based Retrieval Augmented Generation Algorithm

Zihang Li, Wenjun Liu, Yikun Zong, Jiawen Tao, Siying Dai, Songcheng Ren,
Zirui Liu, Yuhang Wang, Yanbing Jiang, Tong Yang[†]

Peking University

[†]Corresponding Author

Abstract

As an important paradigm for enhancing the generation quality of Large Language Models (LLMs), retrieval-augmented generation (RAG) faces the two challenges regarding retrieval accuracy and computational efficiency. This paper presents a novel RAG framework called Bridge-RAG. To overcome the accuracy challenge, we introduce the concept of *abstract* to bridge query entities and document chunks, providing robust semantic understanding. We organize the abstracts into a tree structure and design a multi-level retrieval strategy to ensure the inclusion of sufficient contextual information. While this hierarchical organization substantially improves answer quality, traversing the tree to locate the abstracts that contain a query entity inevitably introduces additional retrieval overhead. To restore retrieval efficiency, we further integrate the Cuckoo Filter in CFT-RAG (Li et al., 2026), which provides $O(1)$ entity lookup and naturally fits the entity-to-abstract pathway of our framework. Extensive experiments show that Bridge-RAG achieves consistent accuracy improvements across all metrics and up to $1.9\times$ faster retrieval compared to structured RAG baselines. All related code is available in the “software” appendix.

1 Introduction

Retrieval-Augmented Generation (RAG) has become an important paradigm for enhancing the generation quality of Large Language Models (LLMs) by integrating external knowledge (Lewis et al., 2020). Existing RAG frameworks differ chiefly in how the external knowledge is organized: at one end of the spectrum, simple text stores keep documents as raw passages and retrieve them by vector similarity; at the other, structured representations—knowledge graphs and entity trees—encode the relations among concepts so that retrieval can surface richer, more coherent evidence than a flat passage list.

RAG systems face two critical challenges regarding retrieval accuracy and computational efficiency. First, in terms of retrieval accuracy, standard RAG frameworks usually rely on vector similarity search, which tends to retrieve passages that are semantically related but contextually fragmented or noisy. This misalignment often results in the generator receiving insufficient or irrelevant evidence, inevitably propagating errors to the final response. Second, a severe efficiency bottleneck persists in structured RAG frameworks, especially for Tree-RAG: once trees deepen and the leaf set climbs into the millions, walking the hierarchy to pin down the entities a query refers to becomes one of the dominant costs in the response time budget. Our empirical results in Table 1 indicate that the retrieval times of standard RAG frameworks like Tree-RAG and Graph-RAG are still quite long. These two challenges hinder the deployment of RAG systems in real-world scenarios that demand high precision and low-latency responses.

This paper proposes Bridge-RAG to effectively address the two challenges above. The core innovation of Bridge-RAG is the *hierarchical abstract bridge tree*: we introduce the concept of *abstract* and organize abstracts into a tree structure that systematically bridges query entities and document chunks, enabling the retrieval of coherent multi-level context. This hierarchical organization, however, increases the cost of locating relevant abstracts at retrieval time. To restore efficiency, Bridge-RAG directly integrates the Cuckoo Filter from the recently published CFT-RAG (Li et al., 2026), which provides $O(1)$ entity lookup—significantly faster than the entity-location step in naive Tree-RAG or Graph-RAG—and naturally fits as the entry point of our entity-to-abstract pathway. The workflow of Bridge-RAG is shown in Figure 1.

Our first key design focuses on enhancing retrieval accuracy from the aspects of semantic depth and contextual breadth. First, we introduce the

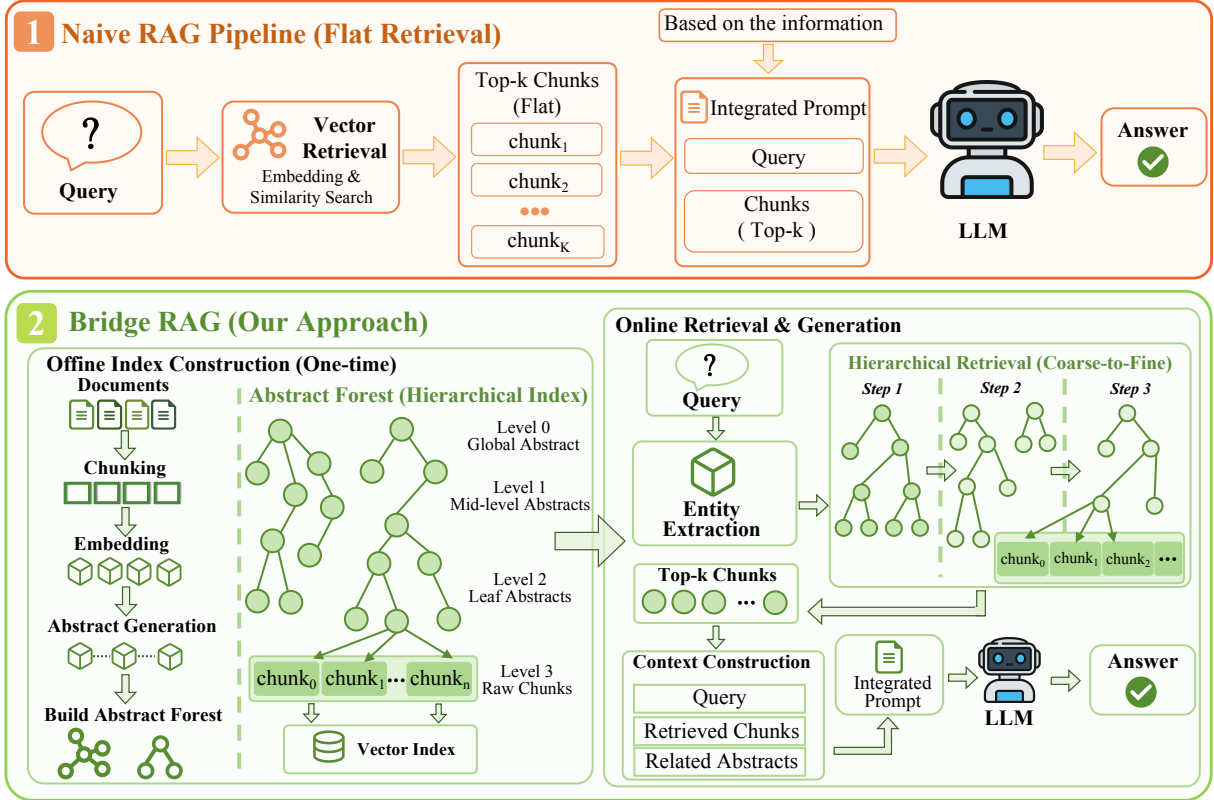


Figure 1: Comparison of Naive RAG and Bridge-RAG. (Top) Naive RAG retrieves flat top- k chunks by similarity, lacking global semantic understanding and cross-document context. (Bottom) Bridge-RAG constructs an abstract forest offline and performs hierarchical coarse-to-fine retrieval online: query entities are located via $O(1)$ Cuckoo Filter lookup, the abstract tree is traversed for multi-level context, and fine-grained chunks are selected for prompt construction and generation.

concept of *abstract* to facilitate robust semantic understanding. Specifically, an *abstract* groups several consecutive document chunks into a high-level knowledge unit. We organize these abstracts into tree structures where upper-level abstracts represent more general concepts and lower-level ones capture more detailed information. This hierarchical organization facilitates a three-tier retrieval path where *abstract* serves as the bridge between *query entity* and *document chunk*. In this way, Bridge-RAG ensures the captured content is semantically relevant. Second, we design a multi-level retrieval strategy to ensure the inclusion of sufficient contextual information. Specifically, Bridge-RAG systematically traces the hierarchical lineage of the abstract tree, retrieving document chunks associated with both parent and child abstracts alongside the target node. This strategy ensures that the retrieved context spans multiple abstraction levels, enabling Bridge-RAG to maintain a comprehensive contextual scope and further improve the answer accuracy.

To make the abstract bridge tree practical at

scale, Bridge-RAG plugs in the Cuckoo Filter recently published in CFT-RAG (Li et al., 2026) as its entity lookup component. Our contribution at this layer is not a new probabilistic data structure but its *integration* into the abstract-tree retrieval path: each filter entry now resolves an entity to a list of abstract-node addresses, so a single filter hit suffices to recover the full entity-to-abstract-to-chunk path, with chunk addresses derived deterministically from the matched abstracts. For the underlying filter mechanics, we refer the reader to CFT-RAG (Li et al., 2026).

Experimental results demonstrate that Bridge-RAG consistently outperforms all baselines on both accuracy and retrieval speed. On MedQA, Bridge-RAG improves ROUGE-L by up to 9.8% and BLEU by up to 11.6% over Tree RAG, while being $1.7\times$ faster. These gains confirm that our *hierarchical abstract bridge tree* effectively captures complex semantic relationships and context, while the integrated Cuckoo Filter from CFT-RAG (Li et al., 2026) sustains high retrieval efficiency as the knowledge base scales.

2 Related Work

Retrieval-Augmented Generation. RAG systems ground LLM outputs in external knowledge to reduce hallucination (Lewis et al., 2020). The simplest variants embed document chunks into a vector store and retrieve the top- k by cosine similarity (Karpukhin et al., 2020). Because flat retrieval treats every chunk independently, it often returns semantically similar but contextually disjoint passages, leaving the generator to reconcile fragments on its own (Gao et al., 2023). Subsequent work has scaled dense retrieval significantly: RETRO (Borgeaud et al., 2022) retrieves from trillions of tokens at pre-training time, and Atlas (Izacard et al., 2023) demonstrates that coupling a retriever with a language model enables strong few-shot performance on knowledge-intensive tasks. Despite these advances, the core limitation of flat retrieval persists: individual passages lack the broader semantic context needed for complex, multi-hop questions.

Structured Knowledge for RAG. To provide richer context, recent work organizes external knowledge into graphs or trees before retrieval. Graph RAG (Edge et al., 2025; Hu et al., 2024) constructs entity–relation graphs and performs multi-hop traversal over node embeddings, effectively capturing relational context. Tree-structured approaches such as T-RAG (Fatehkia et al., 2024) arrange entities hierarchically so that ancestor nodes supply broader context. RAPTOR (Sarathi et al., 2024) builds a recursive abstraction tree by clustering and summarizing chunks bottom-up, enabling retrieval at multiple granularity levels. Bridge-RAG shares RAPTOR’s intuition that intermediate summaries improve retrieval quality, but differs in two key respects: (i) our abstracts are deterministic fixed-window groupings rather than stochastic clusters, which makes the tree structure reproducible and index-friendly; and (ii) we pair the tree with an $O(1)$ Cuckoo Filter entry point rather than relying on embedding-based search at every tree level.

Multi-Hop and Reasoning-Augmented Retrieval. Complex questions often require synthesizing evidence from multiple passages, motivating multi-hop retrieval designs. IRCOT (Trivedi et al., 2023) interleaves chain-of-thought reasoning with retrieval steps, using intermediate reasoning to formulate follow-up queries. HippoRAG (Gutiérrez et al., 2024) models long-term memory inspired

by hippocampal indexing theory, maintaining a knowledge graph that supports multi-hop associative retrieval. Hyper-RAG (Luo et al., 2026) extends graph-based retrieval to hypergraphs for capturing higher-order entity relationships. These approaches focus on the *reasoning* side of multi-hop QA; Bridge-RAG complements them by providing a hierarchical *index structure* that surfaces multi-level context in a single retrieval pass, without requiring iterative query reformulation.

Efficient Retrieval with Probabilistic Data Structures. Approximate membership structures such as Bloom filters (Bloom, 1970) and Cuckoo filters (Fan et al., 2014; Pagh and Rodler, 2001) have long been used to accelerate set-membership queries. CFT-RAG (Li et al., 2026) is the first to embed a Cuckoo Filter into a RAG pipeline, storing entity-to-tree-node mappings for constant-time lookup. Bridge-RAG builds on this line of work but shifts the contribution focus: whereas CFT-RAG uses the filter to locate *entities* in a flat entity tree, we use it to locate *abstracts*—semantic summary nodes that bridge entities and document chunks. The filter itself is adopted from CFT-RAG without modification; our novelty lies in the hierarchical abstract bridge tree and the multi-level context retrieval strategy that operates over it. In summary, the core distinction of Bridge-RAG is the *abstract bridge* layer: by interposing semantic summaries between entities and raw chunks, we convert a flat entity lookup into a structured, multi-granularity retrieval path that neither flat-filter nor graph-traversal approaches provide.

3 Methodology

In this section, we present Bridge-RAG. Its centerpiece is a hierarchical abstract bridge tree that improves retrieval accuracy by bridging query entities, abstracts, and document chunks. To keep the retrieval over this hierarchy efficient at scale, we further integrate the Cuckoo Filter component from CFT-RAG (Li et al., 2026) for $O(1)$ entity-to-abstract lookup.

3.1 Hierarchical Abstract Tree Structure

The foundation of our approach lies in the hierarchical abstract tree structure, which addresses the accuracy limitations of traditional RAG systems. Abstracts are built by semantically grouping n consecutive document chunks into higher-level knowledge units, where n is a configurable window

size (set to 5 in our experiments). Each abstract receives a pair identifier i and subsumes chunk IDs $n_i, n_{i+1}, \dots, n_{i+n-1}$. The n constituent chunks are merged via embedding aggregation or summarization to produce a concise semantic summary, enabling efficient abstraction-layer retrieval.

Abstracts are organized hierarchically into tree structures, where upper-level abstracts represent more general and abstract concepts, while lower-level abstracts capture more specific and detailed information. This hierarchical organization enables multi-level semantic retrieval: when a query entity is identified, we not only retrieve the corresponding abstract and its associated document chunks, but also trace the abstract’s parent-child relationships in the tree. Formally, given an initial abstract set $\mathcal{A}_0 = \{a_i\}$ retrieved from the Cuckoo Filter, we expand it through hierarchical traversal:

$$\mathcal{A}_{\text{expanded}} = \mathcal{A}_0 \cup \bigcup_{a \in \mathcal{A}_0} (\mathcal{P}_d(a) \cup \mathcal{C}_d(a)) \quad (1)$$

where $\mathcal{P}_d(a)$ denotes the set of parent abstracts up to d levels upward, and $\mathcal{C}_d(a)$ denotes the set of child abstracts up to d levels downward. By retrieving these related abstracts along with their corresponding document chunks, we construct a comprehensive context spanning multiple abstraction levels, significantly improving answer accuracy by providing both high-level semantic understanding and granular textual details.

3.2 Efficient Lookup via Cuckoo Filter

Traversing the abstract tree to locate the abstracts that contain a given entity introduces non-trivial overhead. To remove this bottleneck without re-inventing the wheel, we incorporate the Cuckoo Filter component in CFT-RAG (Li et al., 2026), which performs constant-time entity lookup and is well-suited as the entry point of our entity-to-abstract pathway.

Our integration only touches the filter at the interface level: each filter entry, instead of pointing to a flat entity or a raw chunk address, is bound to the abstract-node addresses associated with the entity. Because every abstract with pair_id i deterministically owns chunks n_i through n_{i+n-1} , the document-chunk addresses are recoverable from the matched abstracts, so a single filter hit suffices to expose the full entity-to-abstract-to-chunk path. The resulting retrieval workflow is illustrated in Figure 2; the internal mechanics of the filter (fingerprint hashing, bucket layout, eviction, and

access-frequency reordering) are exactly those of CFT-RAG (Li et al., 2026) and are summarized in Appendix A for completeness.

3.3 Context Generation with Entity-Abstract-Chunk Bridge

Our context generation process leverages the hierarchical abstract tree together with the integrated Cuckoo Filter to achieve both high accuracy and efficiency. The workflow demonstrates a bridging mechanism that connects query entities to abstracts and then to specific document chunks, where abstracts serve as the crucial intermediate layer.

The workflow consists of five steps: (1) **Entity Recognition**: Key entities are identified using entity recognition techniques (e.g., SpaCy). (2) **Entity-to-Abstract Lookup**: For each recognized entity, the integrated Cuckoo Filter from CFT-RAG (Li et al., 2026) is queried, returning the set of abstracts that contain it and updating its access counter. (3) **Hierarchical Context Enrichment**: We trace parent-child relationships to expand the abstract set, retrieving parent and child abstracts up to a configurable maximum depth d . (4) **Abstract-to-Chunk Bridge**: We retrieve all chunks associated with the expanded abstract set, where each abstract is linked to n chunks. (5) **Similarity-based Selection**: We compute cosine similarity between query embedding $\mathbf{q}$ and chunk embeddings $\mathbf{c}_j$:

$$\text{sim}(\mathbf{q}, \mathbf{c}_j) = \frac{\mathbf{q} \cdot \mathbf{c}_j}{\|\mathbf{q}\| \|\mathbf{c}_j\|} \quad (2)$$

We then select the top- k chunks $\mathcal{C}_{\text{top-}k}$ with the highest similarity scores:

$$\mathcal{C}_{\text{top-}k} = \arg \max_{\mathcal{C}' \subseteq \mathcal{C}, |\mathcal{C}'|=k} \sum_{\mathbf{c}_j \in \mathcal{C}'} \text{sim}(\mathbf{q}, \mathbf{c}_j) \quad (3)$$

where $\mathcal{C}$ is the set of all chunks associated with the expanded abstract set. The top- k chunks are then selected, and their corresponding abstracts are retrieved. Finally, this multi-level contextual information (selected chunks and abstracts) is combined with the system prompt and query to form the final prompt for the LLM. The complete workflow is illustrated in Figure 1, and the detailed algorithm is presented in Algorithm 1.

4 Experiments

This section reports the empirical study of Bridge-RAG. We first list the baselines and datasets, then present the main accuracy and retrieval-latency

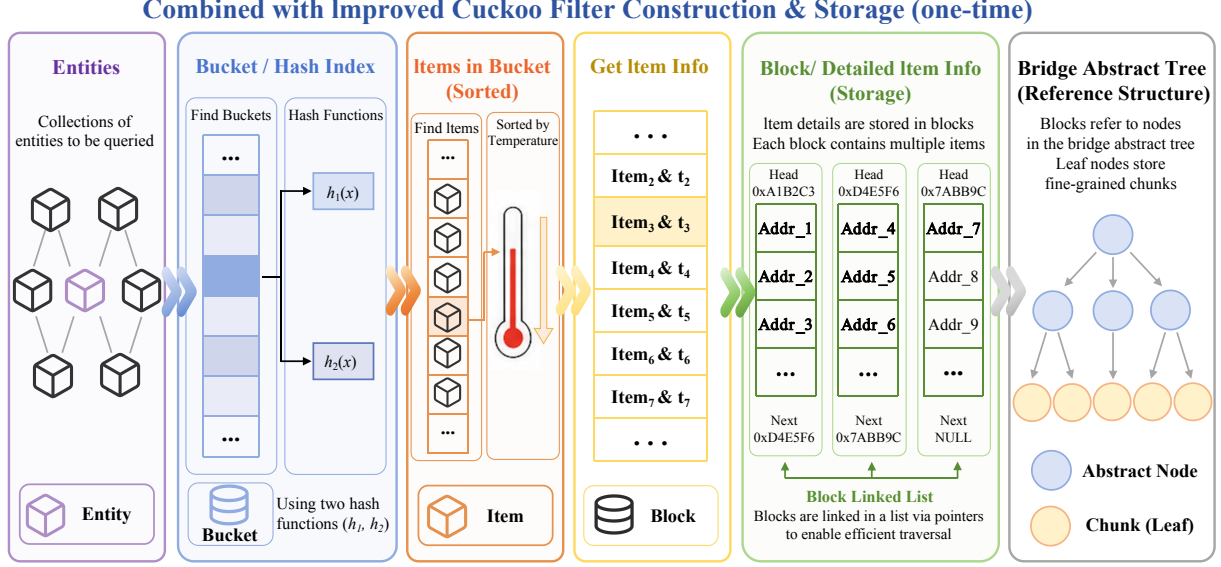


Figure 2: Detailed workflow of Bridge-RAG. Entities recognized from the query are looked up via the Cuckoo Filter in $O(1)$ time. For each matched abstract, the hierarchical context enrichment module traverses parent and child nodes in the abstract tree. The retrieved multi-level context (target, parent, and child abstracts with their chunks) is integrated into a comprehensive prompt for the LLM.

numbers, and finally analyze the effect of the hierarchical depth d and the locality-aware bucket reordering. Our goal is to understand whether the abstract bridge tree pays off in answer quality, and whether the resulting retrieval pipeline stays fast at scale.

4.1 Baseline

We compare Bridge-RAG against three representative RAG variants that span the no-structure, graph-structured, and tree-structured ends of the design space, so that the contribution of our abstract bridge tree can be attributed cleanly.

Naive RAG This baseline follows the standard retrieve-then-generate pipeline without any hierarchical or filtering enhancement. The top- K chunks returned by cosine similarity are directly concatenated with the user query and fed to the LLM. No abstract layer, graph structure, or approximate acceleration is employed, providing a clean reference for measuring the gains introduced by later optimizations.

Graph RAG A graph-based variant in which entities and their typed relations form a directed graph; retrieval is accelerated with approximate nearest neighbor (ANN) indexing over node embeddings, and multi-hop expansion along the graph provides the surrounding context that the generator conditions on. This baseline isolates the gains of graph-style relational expansion from those of

hierarchical abstraction.

Tree RAG An entity-tree variant that uses approximate nearest neighbor (ANN) indices such as FAISS or HNSW in place of exact similarity search, returning top- K candidates per query. This baseline isolates the gains attributable to our abstract bridge tree (which replaces flat entity leaves with multi-level abstract nodes) from those attributable to the underlying tree structure itself.

RAPTOR (Sarathi et al., 2024) A recursive abstractive processing method that clusters document chunks bottom-up and summarizes each cluster to form a tree. At retrieval time, RAPTOR searches across all tree levels by embedding similarity. Unlike Bridge-RAG, RAPTOR relies on stochastic clustering (rather than deterministic grouping) and uses embedding search at every level (rather than $O(1)$ filter lookup), making it less predictable and slower on large corpora.

4.2 Bridge-RAG

Bridge-RAG combines our hierarchical abstract bridge tree for multi-level context retrieval (Section 3.1) with the Cuckoo Filter component from CFT-RAG (Li et al., 2026) for $O(1)$ entity-to-abstract lookup (Section 3.2). The implementation details are provided in the Methodology section.

Algorithm 1: Bridge-RAG Context Generation

Input: Query q , Cuckoo Filter CF , abstract forest $\mathcal{F}$, chunks-per-abstract n , return count k , max depth d

Output: Context string ctx

```
/* Step 1: Entity recognition and
   Cuckoo Filter lookup */
1  $\mathbf{q} \leftarrow \text{EMBED}(q)$ ;
2  $E \leftarrow \text{RECOGNIZEENTITIES}(q)$ ;
3  $\mathcal{A}_0 \leftarrow \emptyset$ ;
4 foreach  $e \in E$  do
5    $\mathcal{A}_0 \leftarrow \mathcal{A}_0 \cup CF.\text{LOOKUP}(e)$ ;
6    $CF.\text{UPDATETEMP}(e)$ ;

/* Step 2: Hierarchical context
   enrichment */
7  $\mathcal{A} \leftarrow \mathcal{A}_0$ ;
8 foreach  $a \in \mathcal{A}_0$  do
9    $\mathcal{A} \leftarrow \mathcal{A} \cup \mathcal{P}_d(a, \mathcal{F}) \cup \mathcal{C}_d(a, \mathcal{F})$ ;

/* Step 3: Abstract-to-chunk bridge
   and ranking */
10  $\mathcal{C}_{\text{all}} \leftarrow \bigcup_{a \in \mathcal{A}} \{n_{a+j} \mid 0 \leq j < n\}$ ;
11  $\mathcal{C}_k \leftarrow \arg \max_{S \subseteq \mathcal{C}_{\text{all}}, |S|=k} \sum_{c \in S} \text{sim}(\mathbf{q}, \text{EMBED}(c))$ ;

/* Step 4: Prompt construction */
12  $ctx \leftarrow \text{BUILDPROMPT}(\mathcal{C}_k, \mathcal{A}, q)$ ;
13 return  $ctx$ ;
```

4.3 Datasets and Abstract Forest

We evaluate on a long-context dataset, MedQA (Jin et al., 2021), and a medium-sized dataset, AALCR (Artificial Analysis Team, 2025). From each corpus we run the pipeline of Section 3.1 to obtain concepts, relations, and a filtered hierarchy; we then group every n consecutive document chunks into one abstract node ($n=5$), which gives us an abstract forest whose leaves expose document content and whose internal nodes expose semantic context. This forest is the data that all Bridge-RAG retrieval queries are executed against.

4.4 Setup

The RAG orchestration code is written in Python, with the latency-sensitive components (the Cuckoo Filter and the forest traversal) implemented in C++. All measurements are taken on a NVIDIA RTX 4090 machine, and each configuration is run

100 times so that the reported numbers are averages over independent trials. We use Doubao Seed 1.6 (Doubao AI, 2024) as the generator, and judge answer quality with the standard suite of ROUGE, BLEU and BERTScore.

4.5 Comparison Experiment

For MedQA we sample 1,000 questions uniformly while 100 questions for AALCR and we report the per-query retrieval latency together with answer-quality scores (ROUGE, BLEU, BERTScore) averaged over the sample. Headline numbers for MedQA and AALCR are collected in Table 1.

Bridge-RAG achieves the best performance across both datasets. On AALCR, Bridge-RAG (Depth=3) improves BLEU by 44.0% over Tree RAG while being $1.67\times$ faster, and by 63.7% over Graph RAG while being $1.77\times$ faster. Compared to Naive RAG, Bridge-RAG improves ROUGE-L by 78.7% and BLEU by 141.0%.

On MedQA, Bridge-RAG improves ROUGE-L by 9.8% and BLEU by 11.6% over Tree RAG, while being $1.72\times$ faster. Compared to Graph RAG, Bridge-RAG improves ROUGE-L by 9.4% and BLEU by 17.2%, while being $1.91\times$ faster. Compared to Naive RAG, Bridge-RAG improves ROUGE-L by 24.8% and BLEU by 20.5%.

Among all baselines, RAPTOR (Sarathi et al., 2024) is the most architecturally similar to Bridge-RAG, as both build hierarchical summary structures over document chunks. However, the two systems differ fundamentally in their design philosophy. RAPTOR uses unsupervised clustering (e.g., Gaussian Mixture Models) to group chunks, producing clusters of variable size that depend on the clustering hyperparameters and random initialization. Bridge-RAG instead uses deterministic fixed-window grouping (every 5 consecutive chunks), which guarantees reproducibility and enables the Cuckoo Filter’s direct address mapping. Empirically, RAPTOR underperforms Bridge-RAG on both datasets (Table 1). On MedQA, Bridge-RAG (Depth=3) outperforms RAPTOR on all metrics, with a 18.0% gain in ROUGE-L and 7.9% in BLEU. On AALCR, the gap narrows on ROUGE-2 (where RAPTOR’s clustering occasionally captures useful cross-chunk correlations), but Bridge-RAG maintains clear leads on ROUGE-1, ROUGE-L, BLEU, and BERTScore.

The speed advantage comes from the integrated Cuckoo Filter’s direct entity lookup ($O(1)$ complexity), compared to Tree-RAG’s exhaustive BFS

Dataset	Method	ROUGE-1	ROUGE-2	ROUGE-L	BLEU	BERTScore	Retrieval Time (s)
AALCR	Naive RAG	0.0890	0.0303	0.0812	0.0178	0.8244	-
	Raptor (Sarathi et al., 2024)	0.1294	0.0533	0.1253	0.0381	0.8279	-
	Tree RAG	0.1339	0.0501	0.1282	0.0298	0.8326	9.6377
	Graph RAG	0.1307	0.0344	0.1259	0.0262	0.8332	10.2088
	Bridge RAG(Depth=1)	0.1214	0.0393	0.1160	0.0390	0.8325	5.7162
	Bridge RAG(Depth=2)	0.1437	0.0527	0.1365	0.0364	0.8402	5.7231
	Bridge RAG(Depth=3)	0.1522	0.0495	0.1451	0.0429	0.8386	5.7564
MedQA	Naive RAG	0.3081	0.1529	0.3048	0.0679	0.8632	-
	Raptor (Sarathi et al., 2024)	0.3382	0.1452	0.3225	0.0758	0.8672	-
	Tree RAG	0.3498	0.1415	0.3466	0.0733	0.8698	8.7238
	Graph RAG	0.3533	0.1412	0.3478	0.0698	0.8712	9.6782
	Bridge RAG(Depth=1)	0.3501	0.1467	0.3452	0.0776	0.8741	5.0237
	Bridge RAG(Depth=2)	0.3599	0.1430	0.3542	0.0722	0.8772	5.0354
	Bridge RAG(Depth=3)	0.3884	0.1678	0.3805	0.0818	0.8784	5.0618

Table 1: Main results on the benchmarks. Bold marks the best performance per dataset.

traversal. Moreover, our method only needs to calculate the similarity between the query and the candidate chunk pool, rather than the full set of chunks, which greatly speeds up the process. The accuracy improvement stems from Bridge-RAG’s explicit hierarchical abstraction levels (parent, target, child) that capture multi-level semantic relationships, providing more global and semantically coherent context than graph-based traversal or direct chunk retrieval. The multi-level context retrieval strategy traces parent-child relationships in the abstract tree, retrieving context from multiple abstraction levels to enable more accurate and contextually coherent answers.

Comparing different depth settings reveals a trade-off between accuracy and efficiency. The accuracy improvement with increasing depth comes from retrieving context from more abstraction levels: Depth=2 and Depth=3 additionally retrieve deeper parent and child abstracts, providing broader semantic context and more granular details. The increased retrieval time is due to the additional tree traversal operations required to access parent and child nodes, but remains efficient thanks to the fast entity-to-abstract mapping that minimizes the overhead of hierarchical expansion.

4.6 Complexity Analysis

Time complexity. Given a query with $|E|$ recognized entities, Bridge-RAG performs (i) $|E|$ Cuckoo Filter lookups, each costing $O(1)$; (ii) hierarchical expansion to depth d , visiting at most $O(b^d)$ abstract nodes per initial hit, where b is the branching factor; and (iii) a top- k similarity selection over the candidate chunk pool $|C|$, costing $O(|C| \cdot \text{dim})$ where dim is the embedding dimension. In practice $b \leq 5$ and $d \leq 3$, so the expansion

is bounded by a small constant. The dominant cost is the similarity computation, which operates over a filtered candidate set rather than the full corpus—typically 2–3 orders of magnitude smaller. By contrast, Tree RAG performs BFS over the entire entity tree ($O(N)$ for N entities), and Graph RAG traverses multi-hop neighborhoods that grow combinatorially with hop count.

Space complexity. The Cuckoo Filter stores one 12-bit fingerprint, one temperature counter, and one block-list pointer per entity, yielding $O(|E_{\text{total}}|)$ space with a small per-entry constant (~ 4 bytes). The abstract forest adds $O(|A|)$ nodes. Overall, Bridge-RAG’s indexing overhead is linear in corpus size and negligible compared to the embedding store.

5 Discussion

Why abstracts outperform flat entities. A key insight from our results is that the abstract layer provides a semantic granularity that neither raw chunks nor flat entities offer. Raw chunks are too fine-grained: they capture local details but miss the broader context needed for multi-hop questions. Flat entities, as used in standard Tree RAG, are too coarse: they index individual concepts without encoding how those concepts relate within a passage. Abstracts occupy a middle ground—each one summarizes a coherent group of chunks and naturally encodes local relationships, making them effective bridges for the entity-to-chunk retrieval path.

Depth selection in practice. Our depth study reveals that Depth=3 consistently achieves the best accuracy, but the marginal gain from Depth=2 to

Depth=3 is smaller than from Depth=1 to Depth=2 (Table 1). This diminishing return suggests that most useful context resides within two levels of the target abstract. For latency-sensitive deployments, Depth=2 offers a practical sweet spot—retaining most of the accuracy benefit at lower traversal cost. Adaptive depth selection, where the system chooses depth based on query complexity, is a promising direction we leave for future work.

Interaction between the abstract tree and the Cuckoo Filter. While Bridge-RAG adopts the Cuckoo Filter from CFT-RAG without modification, the two components are not merely stacked independently. The abstract tree changes the semantics of what the filter indexes: instead of mapping entities to tree nodes in a flat entity hierarchy, each filter entry now maps to an abstract node that deterministically owns chunks and participates in a parent–child hierarchy. This means a single filter hit not only locates the target content but also opens a path to hierarchically related context—an affordance that a flat entity-to-chunk mapping cannot provide.

Cross-dataset generalization. Bridge-RAG’s gains are consistent across MedQA (a domain-specific medical corpus with specialized terminology) and AALCR (a general long-context reasoning benchmark), suggesting that the abstract bridge tree is not tuned to a particular domain. The hierarchical structure is especially beneficial for MedQA, where medical concepts form natural taxonomies (e.g., diseases → symptoms → treatments) that align well with the parent–child organization of the abstract tree. On AALCR, the improvement is driven more by the multi-level context providing broader reasoning scope for complex, multi-hop questions. This cross-domain consistency suggests that the abstract bridge tree captures a general structural pattern rather than domain-specific regularities.

Scalability. Bridge-RAG’s $O(1)$ filter lookup is independent of corpus size, whereas Tree RAG’s BFS scales linearly and Graph RAG’s multi-hop expansion grows combinatorially. Only the final top- k selection over candidate chunks grows with the corpus, and this set is bounded by the small number of matched abstracts. Our $1.7\times\text{--}1.9\times$ speedups over Tree RAG and Graph RAG on AALCR and MedQA should therefore widen further on larger corpora where traversal costs dominate.

When does Bridge-RAG underperform?

Bridge-RAG does not dominate on every metric. On AALCR, RAPTOR achieves higher ROUGE-2 at Depth=1 and Depth=2 (Table 1), suggesting that RAPTOR’s clustering occasionally captures bigram-level patterns that fixed-window grouping misses. Bridge-RAG at Depth=1 also underperforms Graph RAG on BERTScore for AALCR, confirming that the abstract tree’s advantage requires multi-level traversal—the bridge layer alone, without parent–child expansion, provides limited benefit.

6 Conclusion

We presented Bridge-RAG, a retrieval-augmented generation framework whose core contribution is a *hierarchical abstract bridge tree* that connects query entities to document chunks via an intermediate abstract layer, retrieving coherent multi-level context through parent–child traversal. To keep this hierarchical retrieval fast at scale, we delegate the entity lookup step to the Cuckoo Filter of CFT-RAG (Li et al., 2026), which integrates seamlessly with our pipeline. Experiments on MedQA and AALCR show consistent accuracy gains over all baselines (up to 9.8% ROUGE-L and 44% BLEU over structured RAG methods), with $1.7\times\text{--}1.9\times$ faster retrieval than Tree RAG and Graph RAG. Our analysis further shows that the abstract layer provides a semantic granularity unavailable in flat entity or chunk-based retrieval, and that hierarchical depth of 2–3 captures most useful context with bounded overhead. These results demonstrate that bridging the gap between entities and chunks via learned summaries is a promising paradigm for scalable, high-quality RAG.

Looking ahead, we identify three promising directions. First, replacing the fixed five-chunk window with content-aware segmentation could better align abstract boundaries with natural topic shifts. Second, adaptive depth selection—choosing the traversal depth based on query complexity or entity density—could further optimize the accuracy–latency trade-off for heterogeneous workloads. Finally, combining the hierarchical abstract bridge with iterative reasoning strategies such as chain-of-thought retrieval (Trivedi et al., 2023) could unlock stronger performance on multi-hop questions that require sequential evidence gathering across distant parts of the corpus.

Limitations

Bridge-RAG assumes a fixed five-chunk grouping window for abstract construction. While this design enables deterministic indexing and efficient Cuckoo Filter integration, it may not generalize well to all document structures—corpora with highly variable paragraph lengths or non-linear layouts (e.g., tables, dialogue transcripts) could benefit from content-aware or variable-length segmentation. A systematic study of how grouping size affects the accuracy–efficiency frontier is left for future work. Moreover, although the accuracy has been improved, the construction cost of the bridge forest is very high. Reducing both the time and token cost is also included in our future work. Finally, Bridge-RAG inherits the hallucination risk of the frozen backbone LLM: when retrieved chunks themselves contain conflicting or outdated facts, the system can propagate rather than correct the error.

References

- Artificial Analysis Team. 2025. Artificial analysis long context reasoning benchmark (aa-lcr). <https://artificialanalysis.ai/models/long-context>. Accessed: 2026-01-05.
- Burton H Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7):422–426.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, and 1 others. 2022. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, pages 2206–2240. PMLR.
- Doubao AI. 2024. Doubao large language model. <https://www.doubao.com>. Accessed May 2025.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitan, Robert Osazuwa Ness, and Jonathan Larson. 2025. [From local to global: A graph rag approach to query-focused summarization](#). *Preprint*, arXiv:2404.16130.
- Bin Fan, David G. Andersen, Michael Kaminsky, and Michael D. Mitzenmacher. 2014. [Cuckoo filter: Practically better than bloom](#). In *Proceedings of the 10th ACM International Conference on Emerging Networking Experiments and Technologies*, pages 75–88. ACM.
- Masoomali Fatehikia, Ji Kim Lucas, and Sanjay Chawla. 2024. [T-rag: Lessons from the llm trenches](#). *Preprint*, arXiv:2402.07483.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jin Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. Hipporag: neurobiologically inspired long-term memory for large language models. NIPS ’24, Red Hook, NY, USA. Curran Associates Inc.
- Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2024. Grag: Graph retrieval-augmented generation. *arXiv preprint arXiv:2405.16506*.
- Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: few-shot learning with retrieval augmented language models. *J. Mach. Learn. Res.*, 24(1).
- Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. [What disease does this patient have? a large-scale open domain question answering dataset from medical exams](#). *Applied Sciences*, 11(14):6421.
- Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.
- Zihang Li, Yangdong Ruan, Wenjun Liu, Zhengyang Wang, and Tong Yang. 2026. [CFT-RAG: An entity tree based retrieval augmented generation algorithm with cuckoo filter](#). In *The Fourteenth International Conference on Learning Representations*.
- Haoran Luo, Haihong E, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, and Anh Tuan Luu. 2026. [HypergraphRAG: Retrieval-augmented generation via hypergraph-structured knowledge representation](#). In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
- Rasmus Pagh and Flemming Friche Rodler. 2001. Cuckoo hashing. In *Proceedings of the Annual European Symposium on Algorithms (ESA)*, volume 2161, pages 121–133.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. [RAPTOR: Recursive abstractive processing for tree-organized retrieval](#). In *The Twelfth International Conference on Learning Representations*.

Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10014–10037.

A Cuckoo Filter Details

Bridge-RAG adopts the Cuckoo Filter from CFT-RAG (Li et al., 2026) without modification to its internal mechanics. We summarize the key aspects for self-containedness.

Entity Insertion. Each entity is represented by a short fingerprint $f(x)$. Two candidate buckets $i_1 = h(x)$ and $i_2 = i_1 \oplus h(f(x))$ are computed. If either bucket has a free slot, the fingerprint, an initial temperature counter, and the entity’s abstract-address list are stored there. Otherwise, the standard cuckoo eviction routine displaces an existing entry, subject to a fixed kick budget.

Access-Frequency Reordering. Every entity carries a temperature counter $T(e)$ incremented on each successful lookup. The filter periodically sorts entries within each bucket in descending order of $T(e)$, so that frequently accessed entities are found first during the linear per-bucket scan.

Storage Layout. Each bucket slot holds: (i) a 12-bit fingerprint, (ii) the temperature counter at the head of the address chain, and (iii) a pointer to a block linked list whose nodes store abstract pair_ids. From each pair_id i , chunks n_i through n_{i+n-1} are derived deterministically. The resulting layout is shown in Figure 2.

B Evaluation Details

B.1 Prompt Template

Bridge-RAG Prompt Template

System: Answer the question using the provided information.

Information:
<retrieved chunks from top-k similarity search>

Abstracts:
<hierarchical abstract summaries>

Question:
<user query>

Assistant:
<generated answer>

For baseline RAG (without abstract tree), the prompt template is simplified to:

Baseline RAG Prompt Template

System: Answer the question using the provided information.

Information:

<retrieved chunks from vector similarity search>

Question:

<user query>

Assistant:

<generated answer>

C The Use of LLMs

An LLM was used solely as a writing aid during paper preparation, to flag grammar and spelling issues and to suggest sentence-level rewrites for fluency. All technical content, claims, and experimental results were authored and verified by the human authors.